

Oluwatobi Melvyn Mayungbo

Software Engineer, Payments and Risk Systems

melvyn.t@gmail.com | [LinkedIn: linkedin.com/in/oluwatobi-mayungbo-3a567026b](https://www.linkedin.com/in/oluwatobi-mayungbo-3a567026b) | [GitHub: github.com/MelvTheGoat](https://github.com/MelvTheGoat)

SUMMARY

Software engineer building payments and risk systems. Built a double-entry ledger API with idempotency keys, a transactional outbox and 138 tests, a payment reconciliation service, and real-time fraud scoring at 5.1 ms p99. Works in Python, PostgreSQL, Redis and FastAPI.

SKILLS

Engineering and MLOps: Python, PostgreSQL, Redis, FastAPI, Docker, Pytest, SQL, DuckDB, GCP Cloud Run, AWS ECS Fargate, Railway, Prefect, dbt-core, MLflow, Evidently, CI/CD (GitHub Actions), Git, Streamlit, React

Payments and Risk: Double-Entry Ledgers and Idempotency, Payment Reconciliation, Transaction Fraud Detection, Credit Risk Decisioning, Regulatory Compliance (CBN/NDPA), Demand Forecasting, Web3 Fraud Risk Scoring

Machine Learning: PyTorch, Scikit-learn, LightGBM, XGBoost, Deep Learning, Sequence Models (GRU, TCN, Transformers), Model Calibration (Isotonic Regression, Temperature Scaling), SHAP, Feature Engineering

AI and LLM Systems: Retrieval-Augmented Generation (RAG), Hybrid Search (BM25, Dense Embeddings, RRF), LLM Evaluation and Guardrails, Prompt Engineering, Citation Verification, Model Context Protocol (MCP), Hugging Face, ONNX Runtime

Statistics and Causal Inference: Statistical Inference, Experiment Design, Power Analysis, CUPED, Causal Inference (S/T/X-Learners, Causal Forests), Uplift Modeling, Qini and AUUC, Time-Series Backtesting, Logistic Regression, WOE Scorecards

PROJECTS

Production-Grade Double-Entry Ledger API | [GitHub](#)

Tech: Python 3.12, FastAPI, PostgreSQL 16, Redis 7, Alembic, Docker

- Built a double-entry ledger and payments API where correctness under concurrency, retries, partial failure and replay is enforced in the database: integer minor units, every transaction summing to zero through a deferred constraint trigger, and an append-only ledger where corrections are reversals.
- Required an Idempotency-Key on every mutating endpoint so a replayed key returns the original response and creates nothing; events go out through a transactional outbox.
- Benchmarked pessimistic against optimistic locking: on a hot single account pessimistic reached 144.8 tps against 41.3 tps (3.5×) while optimistic shed 46% of writes as conflicts; spread across 16 accounts it was 206.5 tps against 112.6 tps.
- Load tested to find throughput saturating at about 250 to 260 req/s; removing the hot account cut transfer p99 from 1800 ms to 390 ms (4.6×) at 50 users.
- Wrote 138 tests against real PostgreSQL and Redis, including a concurrency test where exactly half of N simultaneous transfers succeed and no balance goes negative.

Sequence-Based Transaction Fraud Detection System | [GitHub](#)

Tech: Python, PyTorch, LightGBM, ONNX Runtime, FastAPI, Docker, SHAP

- Deployed real-time scoring by exporting to ONNX Runtime (2.9× faster than eager PyTorch at batch size 1) and serving via FastAPI, reaching 5.1 ms p99 end-to-end latency against a 50 ms budget, with append-only per-decision audit logging and attention-based attribution of the prior transactions driving each flag.
- Benchmarked all architectures against a strong LightGBM baseline on engineered velocity and deviation features, reporting PR-AUC and precision at fixed review capacity (top 0.1%, 0.5%, 1%) across 3 seeds, with a per-attack-type breakdown covering card testing, account takeover and merchant compromise; sequence models cut expected cost per transaction 27% under an explicit cost model.

Reckon: Payment Reconciliation and Review System | [GitHub](#) | [Live App](#)

Tech: Python, PyTorch, DuckDB, FastAPI, React, Docker, Railway, PostgreSQL

- Built and deployed a payment reconciliation system processing multi-source transaction feeds to detect uncollected revenue, duplicate settlements and payout discrepancies, combining deterministic matching rules with ML-assisted discrepancy resolution.

- Engineered a multi-stage reconciliation pipeline using DuckDB and FastAPI, with temporal join logic and tolerance thresholds to handle asynchronous processor webhooks and bank settlement files.
- Credit Risk Decisioning and Fairness Audit System** | [GitHub](#) | [Live App](#)
Tech: Python, LightGBM, scikit-learn, SHAP, WOE Scorecards, FastAPI, Docker
- Developed a full credit decisioning system producing calibrated default probabilities, cost-optimal approve and decline cutoffs, adverse action reason codes and an append-only audit trail that lets any individual decision be reconstructed months later.

EXPERIENCE

- Machine Learning Instructor, SQI College of ICT** March 2026 to Present
- Teach machine learning and deep learning to student cohorts, covering statistical foundations, classical ML algorithms, neural architectures, sequence models and production evaluation metrics.
 - Instruct students on data analysis, feature engineering and model building with PyTorch, Scikit-Learn, Pandas and NumPy.
- Internal Auditor, NISER** May 2024 to July 2024
- Analysed financial and operational records to identify inconsistencies and irregularities, performing manual anomaly detection across complex transactional datasets.

EDUCATION

- B.Sc. Statistics, University of Ibadan** January 2021 to March 2025
- Professional Diploma in Artificial Intelligence, SQI College of ICT** July 2025 to Present

CERTIFICATIONS

- Google Data Analytics Professional Certificate
- Google Advanced Data Analytics Professional Certificate
- Claude Code In Action